

Generate & Recognize QR Code

This lesson is divided into two parts. The first part will introduce you how to learn to create a QR code. The second part focuses on recognizing the created QR code and then decoding the QR code information through the terminal.

1. Generate QR Code

1.1 Process

First, create an instance object of the QR code tool and set its detailed parameters.

Next, obtain the user's data and fill it into the QR code.

Finally, generate a QR code image based on the data and display it in a window, and save it in the corresponding path.

1.2 Operation Steps

Note: the entered command should be case sensitive and the "Tab" key can be used to fill in key words.

1) Start MentorPi and connect it to VNC.

2) Click on  to open the command line terminal.

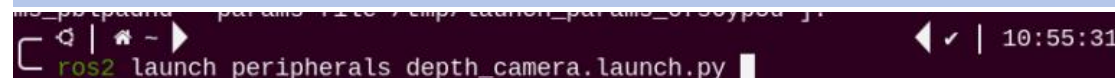
3) Input the command and press Enter to disable the app auto-start service.

```
~/.stop_sh
```



4) Input the command to enable the camera service.

```
ros2 launch peripherals depth_camera.launch.py
```



5) Open a new command line terminal. Enter the following command to navigate to the program directory.

- 3) If need to close the program, press "Ctrl+C" in the terminal. If this fails, please try multiple times.
- 4) After closing the window, the saved content and its path are printed.

```
> python3 qrcode_creator.py
Please enter : hiwonder
save hiwonder /home/ubuntu/ros2_ws/src/example/example/qrcode/myQRcode.jpg
```

1.4 Program Analysis

The source code of the program is located in

/home/ubuntu/ros2_ws/src/example/qrcode/qrcode_creator.py

```
#!/usr/bin/env python3
# encoding: utf-8
import os
import cv2
import qrcode
import numpy as np

def create_qrcode(data, file_name):
    """
    version: 值为1-40的整数，控制二维码的大小（最小值是1，是个12x12的矩阵）。(An integer value between 1 and 40 that controls the size of the QR code (the minimum value is 1, which indicates a 12x12 matrix))
    error_correction: 控制二维码的纠错能力，将值设置为 None 并调用 fit 参数即可。(If you want the program to determine the size automatically, set the value to None and use the fit parameter)
    ERROR_CORRECT_L: 大约7%或更少的错误能被纠正。(can correct about 7% or fewer errors)
    ERROR_CORRECT_M (默认): 大约15%或更少的错误能被纠正。(can correct about 15% or fewer errors)
    ERROR_CORRECT_H: 大约30%或更少的错误能被纠正。(can correct about 30% or fewer errors)
    box_size: 控制二维码中每个小格子包含的像素数。(Controls the number of pixels in each small square in the QR code)
    border: 控制边框（二维码与图片边界的距离）包含的格子数（默认为4，是相关标准规定的最小值）(controls the number of squares (default is 4, which is the minimum value specified by the relevant standard) in the border (the distance between the QR code and the image boundary))
    """
    qr = qrcode.QRCode(
        version=1,
        error_correction=qrcode.constants.ERROR_CORRECT_H,
        box_size=5,
        border=4)
    # 添加数据(add data)
    qr.add_data(data)
    # 填充数据(fill data)
    qr.make(fit=True)
    # 生成图片(generate image)
    img = qr.make_image(fill_color=(0, 0, 0), back_color=(255, 255, 255))
    opencv_img = cv2.cvtColor(np.asarray(img), cv2.COLOR_RGB2BGR)
    while True:
        cv2.imshow('img', opencv_img)
        k = cv2.waitKey(1)
        if k is -1:
            break
    cv2.imwrite(file_name, opencv_img)
    print('save', data, file_name)
```

◆ Creating QR Code Utility Object

Using the qrcode module to create the necessary object and setting various parameters for the QR code.

```
19 qr = qrcode.QRCode(
20     version=1,
21     error_correction=qrcode.constants.ERROR_CORRECT_H,
22     box_size=5,
23     border=4)
```

The parameters of the above function are as follows:

The first parameter "version" is an integer ranging from 1 to 40, used to control the size of the QR code. If you want the program to automatically determine the size, set this value to None and use the "fit" parameter.

The second parameter "error_correction" controls the error correction capability of the QR code, with the following options:

- 1) ERROR_CORRECT_L: can correct approximately 7% or fewer errors.
- 2) ERROR_CORRECT_M: default value, can correct approximately 15% or fewer errors.
- 3) ERROR_CORRECT_H: can correct approximately 30% or fewer

errors.

The third parameter "box_size" controls the number of pixels contained in each small box of the QR code.

The fourth parameter "border" controls the number of boxes included in the border (distance between the QR code and the image boundary), with a default value of 4, which is the minimum value specified by relevant standards.

◆ Generating QR Code

Using the "add_data" and "make" functions to retrieve and fill data, and then using the "make_image" function to generate the image.

```
24 # 添加数据(add_data)
25 qr.add_data(data)
26 # 填充数据(fill data)
27 qr.make(fit=True)
28 # 生成图片(generate image)
29 img = qr.make_image(fill_color=(0, 0, 0), back_color=(255, 255, 255))
```

The parameters of the "make_image" function are as follows:

The first parameter "fill_color=(0, 0, 0)" is the fill color of the image, which is black in this case.

The second parameter "back_color=(255, 255, 255)" is the background color of the image, which is white in this case.

◆ Displaying Image

Converting the color space of the image using the "cvtColor" function, and then displaying it on the window using the "imshow" function.

```
30 opencv_img = cv2.cvtColor(np.asarray(img), cv2.COLOR_RGB2BGR)
31 while True:
32     cv2.imshow('img', opencv_img)
33     k = cv2.waitKey(1)
```

◆ Saving Image

Using the "imwrite" function to store the generated QR code image and print relevant information.

```
36 cv2.imwrite(file_name, opencv_img)
37 print('save', data, file_name)
```

The parameters of the "imwrite" function are as follows:

The first parameter "file_name" is the storage path of the image.

The second parameter "opencv_img" is the image to be stored.

2. QR Code Recognition

In the previous section, we created a QR code. In this section, we'll perform content recognition on the QR code.

2.1 Recognition Process

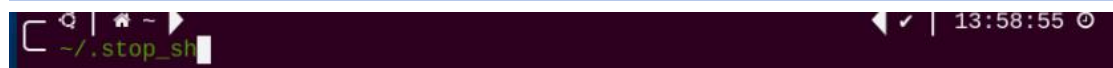
Firstly, create an instance object for QR code detection. Then, subscribe to an image topic “/ascamera/camera_publisher/rgb0/image” in the node and process the received image message. Use the pyzbar library to decode the QR code in the image, and draw the QR code boundary and decoded data using OpenCV. Display the processed image using OpenCV. Next, you can exit the display window by pressing the key.

2.2 Operation Steps

Note: When entering commands, it is necessary to strictly distinguish between uppercase and lowercase letters, and you can use the "Tab" key to complete keywords.

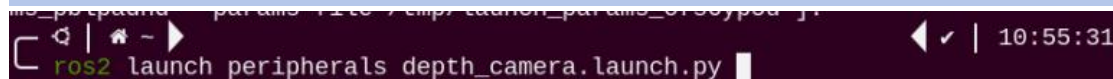
- 1) Start MentorPi and connect it to VNC.
- 2) Input the command and press Enter to disable the app auto-start service.

```
~/stop_sh
```



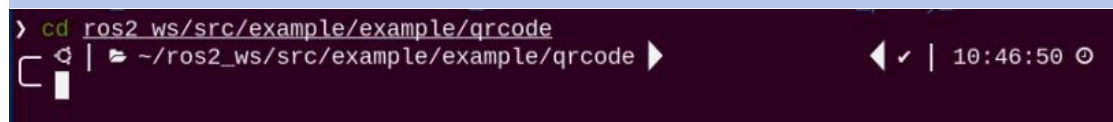
- 3) Run the following command to enable the camera node:

```
ros2 launch peripherals depth_camera.launch.py
```



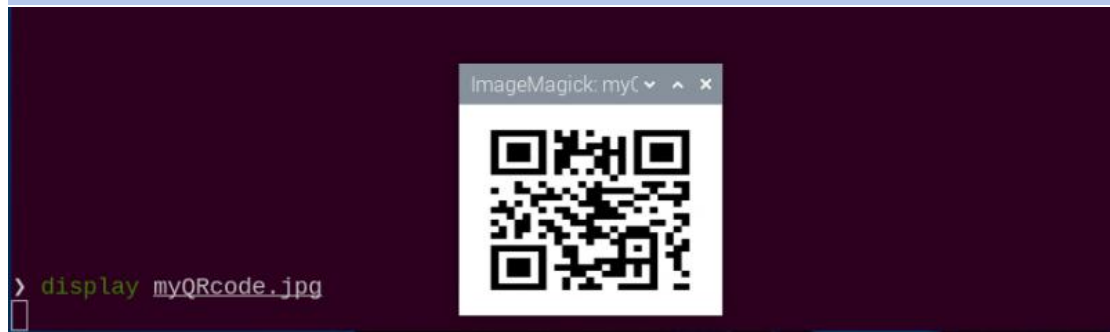
- 4) Open a new command line terminal. Enter the following command to navigate to the program directory.

```
cd ros2_ws/src/example/example/qrcode
```



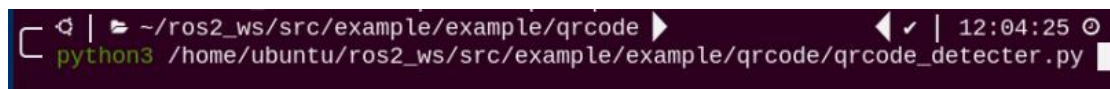
- 5) Enter the command. Press “Enter” to open the image.

```
display myQRcode.jpg
```



6) Open a new terminal and enter the command. Press “Enter” to execute the program.

```
python3 /home/ubuntu/ros2_ws/src/example/example/qrcode/qrcode_detector.py
```



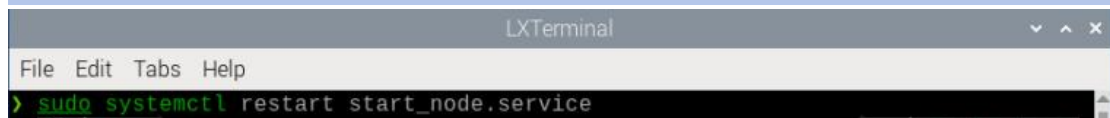
7) If you want to exit the game, press “Ctrl+C” in the terminal interface. After experiencing the game, you can enable the app service through commands or by restarting the robot. If the app is not enabled, the related app functions will not work. If the robot is restarted, the app will be automatically enabled.



Click  and enter the command. Press enter to start the app, and wait for the buzzer to beep.

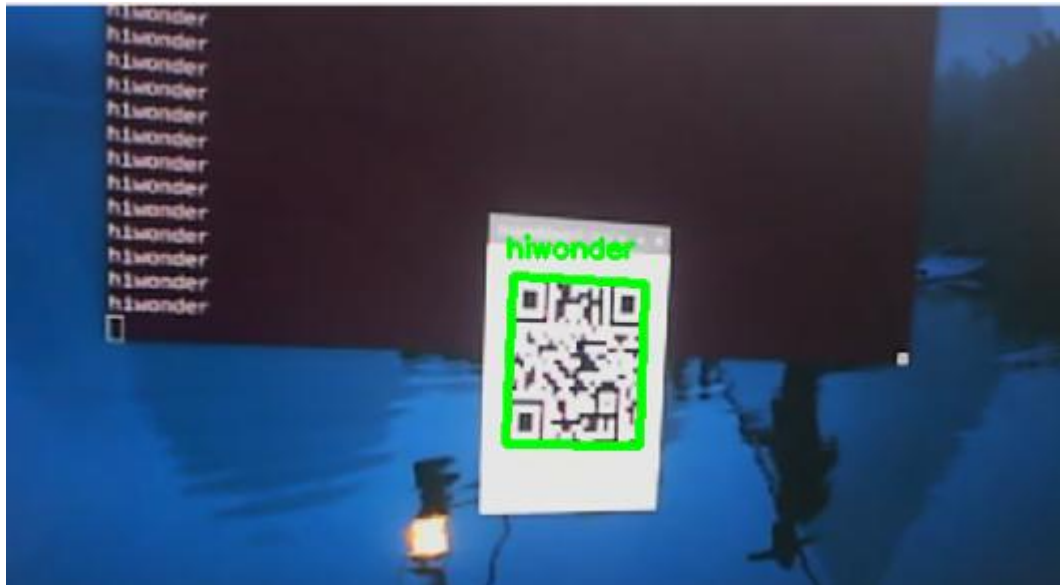
Note: please enter the command in the system path, not in the Docker container.

```
sudo systemctl restart start_node.service
```



2.3 Program Outcome

After starting the game, it will recognize the QR code images appearing in the live camera feed, mark them with a green box, and print out the content of the QR code.



2.4 Program Analysis

The source code of the program is located in

`/home/ubuntu/ros2_ws/src/example/qrcode/qrcode_detector.py`

```
1 #!/usr/bin/env python3
2 # encoding: utf-8
3 import os
4 import queue
5 import rclpy
6 import threading
7 import numpy as np
8 from rclpy.node import Node
9 from cv_bridge import CvBridge
10 from sensor_msgs.msg import Image
11 from pyzbar import pyzbar
12 import cv2
13
14 class QRCodeDetectNode(Node):
15     def __init__(self, name):
16         rclpy.init()
17         super().__init__(name)
18         self.running = True
19         self.bridge = CvBridge()
20         self.image_queue = queue.Queue(maxsize=2)
21         self.image_sub = self.create_subscription(Image, '/camera_publisher/rgb0/image', self.image_callback, 1)
22         threading.Thread(target=self.main, daemon=True).start()
23
24     def image_callback(self, ros_image):
25         cv_image = self.bridge.imgmsg_to_cv2(ros_image, "bgr8")
26         bgr_image = np.array(cv_image, dtype=np.uint8)
27         if self.image_queue.full():
28             self.image_queue.get()
29         self.image_queue.put(bgr_image)
```

◆ Create QR code detection object

Use the pyzbar library to decode the QR code in the image, and draw the QR code boundary and decoded data using OpenCV.

```
41 decoded_objects = pyzbar.decode(image)
42 for obj in decoded_objects:
43     points = obj.polygon
44     if len(points) > 4:
45         hull = cv2.convexHull(np.array([point for point in points], dtype=np.float32))
46         points = hull.reshape(-1, 2)
47         for j in range(len(points)):
48             cv2.line(image, tuple(points[j]), tuple(points[(j+1) % len(points)]), (0, 255, 0), 3)
49         x = obj.rect.left
50         y = obj.rect.top
51         barcode_data = obj.data.decode("utf-8")
52         print(barcode_data)
53         cv2.putText(image, barcode_data, (x, y - 10), cv2.FONT_HERSHEY_SIMPLEX, 0.5, (0, 255, 0), 2)
54
55 cv2.imshow('image', image)
56 key = cv2.waitKey(1)
57 if key == ord('q') or key == 27:
58     break
```

1) “decoded_objects” is a list of QR code objects obtained by decoding the image using `pyzbar.decode(image)`. Each “obj” represents a detected

QR code.

2) Use “cv2.line” to draw the boundary of the QR code. Connect every two consecutive vertices with a line:

- image: the image to draw on.
- tuple(points[j]): the coordinates of the current point.
- tuple(points[(j+1) % len(points)]): the coordinates of the next point, using (j+1) % len(points) to ensure connection to the first point.
- (0, 255, 0): line color, green (BGR format).
- 3: line width.

◆ Get and print the decoded data of the QR code.

```
49 x = obj.rect.left
50 y = obj.rect.top
51 barcode_data = obj.data.decode("utf-8")
52 print(barcode_data)
53 cv2.putText(image, barcode_data, (x, y - 10), cv2.FONT_HERSHEY_
EY_SIMPLEX, 0.5, (0, 255, 0), 2)
```

Use “cv2.putText” to draw the decoded data on the image:

- image: the image to draw on.
- barcode_data: the text to draw.
- (x, y - 10): the coordinates of the bottom-left corner of the text. Subtract 10 pixels to display the text above the QR code.
- cv2.FONT_HERSHEY_SIMPLEX: font type.
- 0.5: font size scale factor.
- (0, 255, 0): text color, green (BGR format).
- 2: text line width.